Supplementary Materials for Fast Density Peaks Clustering Algorithm Based on Approximate *k*-Nearest Neighbors

Shifei Ding, *Senior Member, IEEE*, Chao Li, Xiao Xu, Lili Guo, *Member, IEEE*, Ling Ding, Xindong Wu, *Fellow, IEEE*

S1. Explanation of Overlap Degree Setting

In 3WBKHK, the overlap degree D_{ol} governs the persistence of boundary points across partitioned adjacent subsets, explicitly controlling the count of duplicated boundary points. The methodology for determining boundary points is detailed subsequently.

In BKHK, adequate partitioning depth (sufficient leaf nodes) yields subsets with high internal homogeneity and approximately uniform spatial distribution. As 3WBKHK retains this partitioning structure, its generated subsets consequently inherit the near-uniform distribution property. Assuming that 3WBKHK generates 2^l subsets with k+1 points in each subset, and each point has k neighbors besides itself. Therefore, the number of neighbors of each point is set to k. In a subset with a more uniform distribution, the distance from each point to its k-th nearest neighbor is approximately equal to that of other points. Each subset contains exactly k+1 points, making the distance from the central point to its k-th nearest neighbor the theoretical boundary of the subset. However, for any off-center point in this subset, its distance to the k-th nearest neighbor will be beyond the subset boundary, and the point may be considered as the boundary point of other subsets. Therefore, the boundary points requiring duplication are the points on the side of the line that is perpendicular to the line connecting two centers and passes through one of the centers, such as the green point on the right of line $\bar{a}$ in Fig.1. Since the distribution of points within each subset is approximately uniform, half of the points in each subset are duplicated between adjacent subsets.

Suppose that the number of data points is n and the partition depth is l. For computational ease, let the number of points n' at each partition be divisible by 2. If D_{ol}=0, then each subset generated by 3WBKHK has $n/2^l$ data points. Through the above analysis, $D_{ol}=n/2^{l+1}$ can be obtained. However, $D_{ol}>0$ will increase the number of points in the subset, resulting in the cumulative number of points that need to be divided by 3WBKHK into $n_{cum}=n+D_{ol}\cdot(2+2^2+...+2^l)=n+2(2^l-1)\cdot D_{ol}$.

Dividing n_{cum} points equally into 2^l subsets gives us the number of points in each subset, which is $[n+2(2^l-1)\cdot D_{ol}]/2^l$. Then,

$$\frac{n+2(2^l-1)\cdot D_{ol}}{2^l}=2D_{ol}, \tag{1}$$

The solution gives $D_{ol}=n/2$. However, $D_{ol}=n/2$ makes the same data set after each partition, so direct derivation is not feasible. To address this, we use different D_{ol} and obtain the results in Table I.

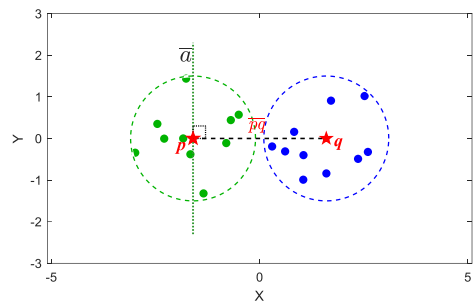

Fig. 1. Two adjacent subsets generated by BKHK. Red stars p and q indicate the two subset centers. The line $\bar{a}$ crossing point p is perpendicular to $\overline{pq}$.

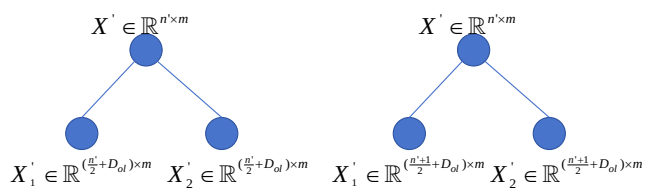

Fig. 2. Presentation of the results of the division of internal node by 3WBKHK. Left: the internal node contains an even number of data points. Right: the internal node contains an odd number of data points.

The ideal case is $n_{cum}/2^l=2D_{ol}$. For Table I, $n_{cum}/2^l>2D_{ol}$. Therefore, smaller $n_{cum}/2^l/D_{ol}$ is better. However, as $n_{cum}/2^l/D_{ol}$ decreases, n_{cum} will increase, increasing the complexity of 3WBKHK. Thus, smaller is better for both $n_{cum}/2^l/D_{ol}$ and n_{cum}. Therefore, $D_{ol}=n/2^{l+1}$ or $n/2^l$ (suboptimal) is chosen, where l is a positive integer large enough to partition the n data points.

The above discussion of the case "if the number of points n' at each partition is divisible by 2" is not comprehensive. As shown in Fig. 2, for any internal node in 3WBKHK, the case where n' is odd adds a data to be processed compared to the case where n' is even. Since 3WBKHK has a total of $(2^0+2^1+...+2^{l-1})$ internal nodes, hence $n_{cum}=n+D_{ol}\cdot(2+2^2+...+2^l)+\lambda=n+2(2^l-1)\cdot D_{ol}+\lambda$, where λ ($0\leq\lambda\leq 2^l-1$) denotes the number of internal nodes containing an odd number of data points.

TABLE I
THE EFFECT OF DIFFERENT D_{ol} ON n_{cum}

D_{ol}	$\frac{n}{2^{l+1}}$	$\frac{n}{2^{l}}$	$\frac{n}{2^{l-1}}$	$\frac{n}{2^{l-2}}$
n_{cum}	$2n-\frac{n}{2^l}$	$3n-\frac{n}{2^{l-1}}$	$5n-\frac{n}{2^{l-2}}$	$9n-\frac{n}{2^{l-3}}$
$n_{cum}/2^l$	$\frac{2n}{2^l}-\frac{n}{2^{2l}}$	$\frac{3n}{2^l}-\frac{2n}{2^{2l}}$	$\frac{5n}{2^l}-\frac{4n}{2^{2l}}$	$\frac{9n}{2^l}-\frac{8n}{2^{2l}}$
$n_{cum}/2^l/D_{ol}$	$4-\frac{1}{2^{l-1}}$	$3-\frac{1}{2^{l-1}}$	$2.5-\frac{1}{2^{l-1}}$	$2.25-\frac{1}{2^{l-1}}$
$n_{cum}^2/2^l/D_{ol}$	$8n-\frac{8n}{2^l}+\frac{n}{2^{2l-1}}$	$9n-\frac{12n}{2^l}+\frac{2n}{2^{2l-1}}$	$12.5n-\frac{20n}{2^l}+\frac{4n}{2^{2l-1}}$	$20.25n-\frac{36n}{2^l}+\frac{8n}{2^{2l-1}}$

TABLE II
THE EFFECT OF DIFFERENT D_{ol} ON n_{cum} IN THE CASE OF λ INTERNAL NODES WITH AN ODD NUMBER OF DATA POINTS

D_{ol}	$\frac{n}{2^{l+1}}$	$\frac{n}{2^{l}}$	$\frac{n}{2^{l-1}}$	$\frac{n}{2^{l-2}}$
n_{cum}	$2n-\frac{n}{2^l}+\lambda$	$3n-\frac{n}{2^{l-1}}+\lambda$	$5n-\frac{n}{2^{l-2}}+\lambda$	$9n-\frac{n}{2^{l-3}}+\lambda$
$n_{cum}/2^l$	$\frac{2n}{2^l}-\frac{n}{2^{2l}}+\frac{\lambda}{2^l}$	$\frac{3n}{2^l}-\frac{2n}{2^{2l}}+\frac{\lambda}{2^l}$	$\frac{5n}{2^l}-\frac{4n}{2^{2l}}+\frac{\lambda}{2^l}$	$\frac{9n}{2^l}-\frac{8n}{2^{2l}}+\frac{\lambda}{2^l}$
$n_{cum}/2^l/D_{ol}$	$4-\frac{1}{2^{l-1}}+\frac{2\lambda}{n}$	$3-\frac{1}{2^{l-1}}+\frac{\lambda}{n}$	$2.5-\frac{1}{2^{l-1}}+\frac{\lambda}{2n}$	$2.25-\frac{1}{2^{l-1}}+\frac{\lambda}{4n}$
$n_{cum}^2/2^l/D_{ol}$	$8n-\frac{8n}{2^l}+\frac{n}{2^{2l-1}}$ $+8\lambda-\frac{\lambda}{2^{l-2}}+\frac{2\lambda^2}{n}$	$9n-\frac{12n}{2^l}+\frac{2n}{2^{2l-1}}$ $+6\lambda-\frac{\lambda}{2^{l-2}}+\frac{\lambda^2}{n}$	$12.5n-\frac{20n}{2^l}+\frac{4n}{2^{2l-1}}$ $+5\lambda-\frac{\lambda}{2^{l-2}}+\frac{\lambda^2}{2n}$	$20.25n-\frac{36n}{2^l}+\frac{8n}{2^{2l-1}}$ $+4.5\lambda-\frac{\lambda}{2^{l-2}}+\frac{\lambda^2}{4n}$
$n_{cum}^2/2^l/D_{ol}(\lambda=n)$	$18n-\frac{12n}{2^l}+\frac{n}{2^{2l-1}}$	$16n-\frac{16n}{2^l}+\frac{2n}{2^{2l-1}}$	$18n-\frac{24n}{2^l}+\frac{4n}{2^{2l-1}}$	$25n-\frac{40n}{2^l}+\frac{8n}{2^{2l-1}}$

Table II shows the effect of different D_{ol} on n_{cum} under parameter λ. According to the analysis procedure in Table I, it is necessary to determine which D_{ol} corresponds to the smallest $n_{cum}^2/2^l/D_{ol}$. In Table II, as D_{ol} increases, $n_{cum}^2/2^l/D_{ol}$ is mainly affected by two parameters, n and λ, while l is set to a positive integer large enough to divide the data adequately. As D_{ol} increases, it can be seen from (2)-(4) that λ causes $n_{cum}^2/2^l/D_{ol}$ to decrease, which is contrary to the upward trend of $n_{cum}^2/2^l/D_{ol}$ in Table I. The larger λ is, the more pronounced this downward trend is. However, according to the parameter settings in Table III, except for seeds, the λ ($\leq 2^l$-1) will not exceed the data size n. Therefore, substituting $\lambda=n$ into the $n_{cum}^2/2^l/D_{ol}$ in Table II, we can assume that when $D_{ol}=n/2^l$, $n_{cum}^2/2^l/D_{ol}$ is minimum.

$$8\lambda-\frac{\lambda}{2^{l-2}}+\frac{2\lambda^2}{n}-\left(6\lambda-\frac{\lambda}{2^{l-2}}+\frac{\lambda^2}{n}\right)=2\lambda+\frac{\lambda^2}{n}, \quad (2)$$

$$6\lambda-\frac{\lambda}{2^{l-2}}+\frac{\lambda^2}{n}-\left(5\lambda-\frac{\lambda}{2^{l-2}}+\frac{\lambda^2}{2n}\right)=\lambda+\frac{\lambda^2}{2n}, \quad (3)$$

$$5\lambda-\frac{\lambda}{2^{l-2}}+\frac{\lambda^2}{2n}-\left(4.5\lambda-\frac{\lambda}{2^{l-2}}+\frac{\lambda^2}{4n}\right)=0.5\lambda+\frac{\lambda^2}{4n}, \quad (4)$$

Overall, there are two candidates for D_{ol} ($\lceil n/2^l \rceil$ or $\lceil n/2^{l+1} \rceil$, $\lceil\cdot\rceil$ ensures that the overlap degree is an integer). $D_{ol}=\lceil n/2^l \rceil$ or $\lceil n/2^{l+1} \rceil$ will be determined by the experimental results under different parameters.

S2. Comparison of Running Times

This section presents a comparison of the running times of all algorithms, as shown in Table IV. On small datasets, such as wine, sonar, and seeds, SMMP's running time is relatively shorter than those of others, while FDPAN's time is at a high level among all algorithms. However, on these datasets, the times of all algorithms are less than 0.05 seconds except for DBSCAN-DPC, indicating high overall efficiency.

TABLE III
THE PARAMETERS ON DIFFERENT DATASETS

Datasets	Size	l	kv	2^l-1
wine	178	5	3	31
sonar	208	4	1	15
seeds	210	8	1	255
pima	768	7	13	127
segmentation	2310	9	13	511
abalone	4177	11	5	2047
ringnorm	7400	8	85	255
pendigits	10992	8	5	255
Gamma	19020	12	7	4095
avila	20867	12	11	4095
madelon	2000	9	10	511
Gisette	7000	8	9	255
CIFAR100	60000	14	10	16383
FMNIST	70000	15	6	32767
ijcnn	141691	13	53	8191
Covertype	581012	14	29	16383

TABLE IV
COMPARISON OF RUNNING TIMES ON REAL DATASETS (IN SECONDS)

Datasets	DPC	DCDP -ASC	MDPC+	KS-FDPC	SMMP	DBSCAN -DPC	NDP-Kmeans	Fast LDP -MST	FDPAN
wine	0.0030	0.0286	0.0177	0.0264	**0.0020**	0.4188	0.0091	0.0041	0.0233
sonar	0.0055	0.0134	0.0027	0.0044	**0.0024**	0.4809	0.0103	0.0049	0.0299
seeds	0.0046	0.0103	0.0054	0.0213	**0.0018**	0.4146	0.0105	0.0024	0.0475
pima	0.0244	0.0470	0.0080	0.0923	0.0105	1.4340	0.0391	**0.0047**	0.0458
segmentation	0.2154	0.1830	0.0714	0.7590	0.0480	6.0627	0.1584	**0.0278**	0.1285
abalone	0.8000	0.7954	0.0512	0.1256	0.0880	8.0219	0.1858	**0.0200**	0.3168
ringnorm	2.6092	5.9942	0.7446	**0.3724**	0.5082	17.1616	2.3530	0.4468	0.5607
pendigits	5.8811	3.8755	1.9764	6.7714	1.4077	24.7226	0.9135	**0.3149**	0.9646
Gamma	18.3561	7.0672	0.2915	16.1794	0.5576	61.2235	1.7240	**0.0729**	0.8839
avila	21.5455	10.0510	0.5156	1.5620	0.9256	85.8534	1.6745	**0.3483**	0.9909
madelon	0.2279	1.7710	**0.1205**	0.1606	0.1384	3.6470	0.3918	0.2030	0.3208
Gisette	19.6665	236.3202	10.6282	20.1491	17.7743	32.3896	40.1080	12.3130	**9.9872**
CIFAR100	-	-	-	-	-	-	2475.8979	155.7868	**112.5481**
FMNIST	-	3847.6224	-	368.5576	-	-	679.2483	70.8865	**48.6788**
ijcnn	-	525.0029	-	-	-	-	47.3589	**5.3485**	6.8885
Covertype	-	4077.4805	-	-	-	-	410.3687	172.5630	**61.5564**

Bold indicates the optimal value and '-' indicates no result.

With the increase of data size, the efficiency improvements of Fast LDP-MST and FDPAN become increasingly evident. On pima, segmentation, abalone, pendigits, Gamma, and avila, Fast LDP-MST always has the lowest time consumption. The efficiency of FDPAN is also gradually higher than those of DPC, DCDP-ASC, KS-FDPC, and NDP-Kmeans. The hybrid integration of distinct KNN strategies in MDPC+, SMMP, and Fast LDP-MST sustains higher computational efficiency.

With further increases of data size and dimension (CIFAR100, FMNIST, ijcnn, and Covertype), the efficiency of FDPAN achieves optimality among all algorithms, except for the running time of FDPAN is slightly higher than that of Fast LDP-MST on ijcnn. The matrix operations used by MDPC+ and SMMP consume excessive memory on these large-scale high-dimensional datasets, and the programs cannot run.

In addition to Fast LDP-MST and FDPAN, NDP-Kmeans can also output clustering results on four large-scale high-dimensional datasets, but the kd-tree is far less efficient than the approximate KNN used by Fast LDP-MST and FDPAN. On CIFAR100 and FMNIST, NDP-Kmeans needs to increase the upper limit of the maximum k when calculating the natural characteristic value to ensure operation. However, this will increase the time of NDP-Kmeans.

While DCDP-ASC successfully processes FMNIST, ijcnn, and Covertype, it exhibits significantly higher computational time versus benchmarked algorithms. This inefficiency stems from: 1) kd-tree's suboptimal performance on large-scale high-dimensional data, and 2) the $O(sn^3)$ complexity of spectral clustering where sn (scaling with data size) denotes initial sub-clusters. Similarly, KS-FDPC handles the FMNIST dataset, but it is very inefficient compared with Fast LDP-MST and FDPAN.

S3. Analysis of Resource Usage

To explore the resource usage in real applications, we further analyze FDPAN. First, given a dataset $X \in \mathbb{R}^{n \times m}$ with two separable Gaussian distributions, we track the resource usage of FDPAN under different n and m, where n and m denote size and dimension, respectively. These benchmark statistics provide quantitative references for assessing computational resource consumption in practical deployments. The metrics used are CPU time and memory, as shown in Tables V and VI, respectively.

TABLE V
CPU TIME CONSUMED BY DATA WITH DIFFERENT SIZES AND DIMENSIONS (S)

	n=2^{17}	n=2^{18}	n=2^{19}	n=2^{20}
m=10	12.4450	26.6900	57.0220	128.5515
m=20	12.6323	27.2297	60.4540	142.8280
m=40	15.2697	32.5183	71.8523	168.7263
m=80	18.8323	39.5990	87.6447	219.1190
m=160	24.1830	55.7600	137.7007	254.0993
m=320	30.2347	68.0407	161.7940	346.0257
m=640	49.4483	107.7087	252.1647	582.4420

TABLE VI
MEMORY CONSUMED BY DATA WITH DIFFERENT SIZES AND DIMENSIONS (GB)

	n=2^{17}	n=2^{18}	n=2^{19}	n=2^{20}
m=10	0.0470	0.1279	0.3683	0.9905
m=20	0.0587	0.1612	0.4197	1.1319
m=40	0.0803	0.2056	0.5468	1.4305
m=80	0.1350	0.3327	0.8208	2.1146
m=160	0.2869	0.6857	1.6485	3.2030
m=320	0.5869	1.3095	3.0229	6.2861
m=640	1.2026	2.6439	5.8860	12.9284

Look for a functional model that can take n and m as input and output the values shown in Tables V and VI. Experimental

observation reveals that CPU time and memory consumption exhibit sublinear growth scaling with a doubling of *m*, initially increasing gradually before entering an accelerated phase. This pattern may be a power or exponential growth, but not too fast. As *n* is doubled, the CPU time and memory also grow to 2-3 times the original, possibly including the power relationship. To this end, an empirical function is used for fitting, and its form is as follows:

$$f(n,m) = a \cdot n^b \cdot m^c, \quad (5)$$

where *a*, *b* and *c* are undetermined coefficients.

From the results in Tables V and VI, the fitting functions of CPU time and memory under *n* and *m* are shown in (6) and (7), respectively.

$$f(n,m) = \exp(-12.1791) \cdot n^{1.1621} \cdot m^{0.3438}, \quad (6)$$

$$f(n,m) = \exp(-20.0895) \cdot n^{1.2944} \cdot m^{0.7135}. \quad (7)$$

Tables V and VI provide a reference basis for the resource usage of FDPAN under real applications. When the size and dimension of the real data are obtained, the resource usage is roughly estimated by looking up the values in Tables V and VI or the changing trend of the value, providing a feasibility analysis.

S4. Illustration of The Convergence of 3WBKHK

Since the objective function of 3WBKHK is proposed based on that of BKHK, we first provide the convergence of BKHK. To simplify the calculation, we adopt the unreduced version of (8), which is (9), for illustration.

$$\min_{g\in\{0,1\},\mathbf{1}^{\mathrm{T}}g=\kappa} g^{\mathrm{T}}(e_1 - e_2), \quad (8)$$

where $g \in \{0,1\}^n$ is the indicator vector ($g_i = 1$ means point x_i belongs to the first cluster, otherwise it belongs to the second cluster). $\mathbf{1}$ is the all-ones vector and $\kappa = \lfloor n/2 \rfloor$ is the constraint on the number of points in the first cluster. e_1 and e_2 are distance vectors, which are the squared Euclidean distances between all points and the two cluster centers c_1 and c_2.

$$J(g, c_1, c_2) = \sum_{i=1}^{n} \sum_{k=1}^{2} g_{ik} \|x_i - c_k\|_2^2, \quad (9)$$

where $g_{i1} + g_{i2} = 1$.

We show that BKHK converges through the following two steps.

1) Monotonically decreasing property of (9)

Allocation step (fix c_1, c_2, and optimize g):

Given fixed cluster centers, the analytical solution of (9) equals that of (8) (selecting the smallest κ components of e_1-e_2) and ensures that the update of g strictly minimizes (9). Therefore,

$$J(g^{(t+1)}, c_1^{(t)}, c_2^{(t)}) \le J(g^{(t)}, c_1^{(t)}, c_2^{(t)}),$$

where $g^{(t)}$ is the value of g at the *t*-th iteration. The equality holds if and only if the allocation does not change anymore.

Center-update step (fix g, and optimize c_1, c_2):

Given a fixed g, the optimal cluster center is the mean of all points in the cluster,

$$c_k^{(t+1)} = \frac{\sum_{i=1}^{n} x_i g_{ik}^{(t+1)}}{\sum_{i=1}^{n} g_{ik}^{(t+1)}}, k=1,2.$$

For the squared Euclidean distance, the mean is the global optimal solution, so

$$J(g^{(t+1)}, c_1^{(t+1)}, c_2^{(t+1)}) \le J(g^{(t+1)}, c_1^{(t)}, c_2^{(t)}).$$

The equality holds if and only if the center does not change.

In summary, each complete iteration satisfies:

$$J^{(t+1)} \le J^{(t)}.$$

That is, the function *J* is a nonincreasing sequence.

2) Lower bound of the objection function (9)

$J \ge 0$, and $J=0$ when all points coincide with the cluster centers.

Since $\{J^{(t)}\}$ is a nonincreasing sequence and has a lower bound, by the monotone convergence theorem, $\{J^{(t)}\}$ converges. Therefore, BKHK converges.

Similar to BKHK, one only needs to show that 3WBKHK is monotonically bounded to conclude its convergence. The objective function of 3WBKHK is (10).

$$\min_{g_1,g_2\in\{0,1\},\mathbf{1}^{\mathrm{T}}g_1=\mathbf{1}^{\mathrm{T}}g_2=\alpha} g_1^{\mathrm{T}}(e_1 - e_2) + g_2^{\mathrm{T}}(e_2 - e_1), \quad (10)$$

where g_1 and g_2 are the indicator vectors ($g_{1i} = 1$ means point x_i belongs to the first cluster, and $g_{2i} = 1$ means point x_i belongs to the second cluster). $\alpha = \lfloor n/2 \rfloor + D_{ol}$ is the constraint on the number of points in each cluster and D_{ol} is the degree of overlap. The remaining symbols are the same as those of (8). According to the derivation process of BKHK in the original paper, we further change (10) into (11).

$$f(g_1, g_2, c_1, c_2) = (g_1 - g_2 + \mathbf{1})^{\mathrm{T}} e_1 + (g_2 - g_1 + \mathbf{1})^{\mathrm{T}} e_2, \quad (11)$$

The two terms in (11) are greater than or equal to 0, so a lower bound exists.

In allocation step of (11), c_1 and c_2 are fixed, so e_1 and e_2 are fixed. The analytical solution of (11) equals that of (10) (selecting the smallest or largest α components of e_1-e_2) and ensures that the update of g_1 or g_2 strictly minimizes (11). Therefore,

$$f(g_1^{(t+1)}, g_2^{(t+1)}, c_1^{(t)}, c_2^{(t)}) \le f(g_1^{(t)}, g_2^{(t)}, c_1^{(t)}, c_2^{(t)}),$$

The equality holds if and only if the allocation does not change anymore.

In center-update step of (11), the optimal cluster can be obtained by taking the partial derivative of c_k (k=1,2) equal to 0, as shown in (12).

$$c_1 = \frac{\sum_{i=1}^{n}(g_{1i} - g_{2i} + 1)x_i}{\sum_{i=1}^{n}(g_{1i} - g_{2i} + 1)}, c_2 = \frac{\sum_{i=1}^{n}(g_{2i} - g_{1i} + 1)x_i}{\sum_{i=1}^{n}(g_{2i} - g_{1i} + 1)}. \quad (12)$$

Let $C_1^{+}=\{i|\, g_{1i} = 1, g_{2i} = 0\}$ and $C_1^{B}=\{i|\, g_{1i} = g_{2i} = 1\}$ represent the positive and boundary regions of the first cluster respectively, then (12) can be rewritten as

$$c_1 = \frac{2\sum_{i\in C_1^+} x_i + \sum_{i\in C_1^B} x_i}{2|C_1^+| + |C_1^B|}, c_2 = \frac{2\sum_{i\in C_2^+} x_i + \sum_{i\in C_2^B} x_i}{2|C_2^+| + |C_2^B|}, \quad (13)$$

where $C_1^B = C_2^B$. However, 3WBKHK uses the same center update formula as BKHK instead of (12), as shown (14).

$$c_1 = \frac{\sum_{i\in C_1^+} x_i + \sum_{i\in C_1^B} x_i}{|C_1^+| + |C_1^B|}, c_2 = \frac{\sum_{i\in C_2^+} x_i + \sum_{i\in C_2^B} x_i}{|C_2^+| + |C_2^B|}. \quad (14)$$

Although (14) will bias centers c_1 and c_2 toward the shared boundary region C_1^B, it will not affect the allocation of the positive regions of two clusters, and the boundary region exists simultaneously in the two clusters. Therefore, the center updates of (13) and (14) will result in the same cluster allocation. It's just that the center update of (14) is not the optimal solution and may converge slightly more slowly.

In addition to the above analysis, we test how the function values of BKHK and 3WBKHK change on all real datasets, as shown in Fig. 3. Both algorithms exhibit relatively consistent convergence, achieving stability within 10 iterations. Only on segmentation, pendigits, and Gisette, 3WBKHK needs more

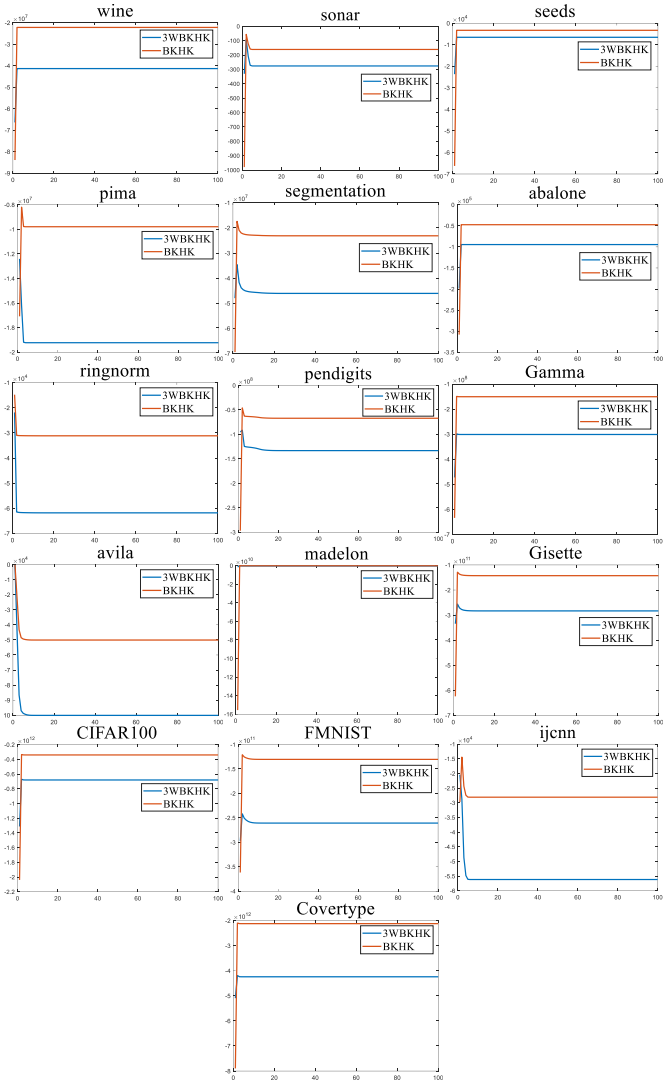

Fig. 3. Convergence comparison of BKHK and 3WBKHK.

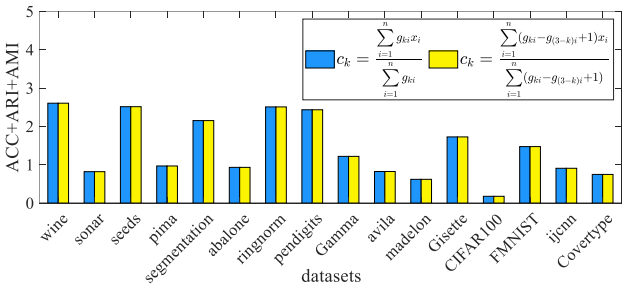

Fig. 4. The effect of two center update methods on the results of FDPAN.

iterations to converge, but it is also within the acceptable range. Moreover, two methods discussed above for center updating in 3WBKHK make FDPAN achieve the same result under the same parameters, as shown in Fig. 4 where k=1,2.

S5. Comparison of the Standard Deviations of Different Densities

Density clustering relies on density separability. If the local densities of all points are very uniform, it is not possible to segment sub-clusters by low-density regions. In FDPAN, the local density on the low-dimensional data is determined by the Euclidean distance, while the density on the high-dimensional data equals the number of reverse k-nearest neighbors. Table VII statistics the standard deviations of two local densities of FDPAN on four high-dimensional datasets, where ρ_1 denotes the first density calculation method in (5) in the main text, ρ_2 represents the second method, and l is the depth of binary tree. Table VII shows that local density calculated by the number of reverse k-nearest neighbors (that is ρ_2) exhibits higher standard deviation, facilitating the identification of low-density regions to partition sub-clusters. Although the number of k-nearest neighbors decreases as l increases and the number of reverse k-nearest neighbors tends to be equal, the standard deviation of density ρ_2 still has a larger value.

TABLE VII
COMPARISON OF THE STANDARD DEVIATIONS OF DIFFERENT DENSITIES

madelon					
l=	6	7	8	9	10
FDPAN-ρ_1	0.000	0.000	0.000	0.000	0.000
FDPAN-ρ_2	59.183	30.650	14.590	6.632	3.177
Gisette					
l=	8	9	10	11	12
FDPAN-ρ_1	0.000	0.000	0.000	0.000	0.000
FDPAN-ρ_2	46.453	22.847	11.262	6.461	3.170
CIFAR100					
l=	11	12	13	14	15
FDPAN-ρ_1	0.000	0.000	0.000	0.001	0.002
FDPAN-ρ_2	76.406	39.482	21.967	11.194	5.371
FMNIST					
l=	12	13	14	15	16
FDPAN-ρ_1	0.001	0.002	0.004	0.008	0.014
FDPAN-ρ_2	34.697	18.044	10.262	6.237	4.098